

The Curious Backend Developer

Expression Trees

1. إحنا بنحاول نفهم إيه أصلاً؟

خلينا نبدأ من سؤال بسيط جداً:

لما تكتب:

```
c => c.Age > 18
```

إيه اللي حصل؟

أغلبنا في البداية بيبص عليها ويقول:

"دي lambda بتاخد customer وترجع true أو false".

وده صح... لكن مش دايماً الصورة الكاملة.

لأن نفس الـ lambda ممكن تستخدمها بطريقتين مختلفتين جداً.

ممكن تقول:

```
Func<Customer, bool> predicate =  
    c => c.Age > 18;
```

وهنا أنت بتقول:

"أنا عايز behavior أقدر أنفذه".

وممكن تقول:

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18;
```

وهنا أنت بتقول:

"أنا عايز representation للـ behavior ده".

ودي أهم فكرة في الـ PDF كله:

Lambda



executable behavior ممكن تبقى

أو

structured representation ممكن تبقى

وده هو السبب الحقيقي اللي يخلينا نهتم بـ Expression Trees.

2. ال Func بتقول "نفذ"، و Expression بتقول "افهم"

شوف المثال ده:

```
Func<int, bool> predicate =  
    x => x > 10;
```

إنت دلوقتي عندك `predicate`.

ممكن تعمل:

```
var result = predicate(15);
```

الـ runtime محتاج ينفذ:

```
x > 10
```

على القيمة:

```
15
```

فالنتيجة:

```
true
```

هنا الـ lambda اتحولت لحاجة قابلة للتنفيذ.

لكن:

```
Expression<Func<int, bool>> expression =  
    x => x > 10;
```

إنت مش بتطلب منه فقط behavior قابل للتنفيذ.

إنت بتطلب representation للكود.

يعني بدل:

```
"x > 10" شغل
```

بقى عندك مفهوم أقرب إلى:

```
"x > 10" ده شكل التعبير
```

وده فرق هندسي ضخمة.

3. طب يعني إيه "Representation"؟

دي النقطة اللي لازم تثبت في دماغك.

لما تقول:

```
x => x > 10
```

ممكن تمثيلها في دماغك كـ text:

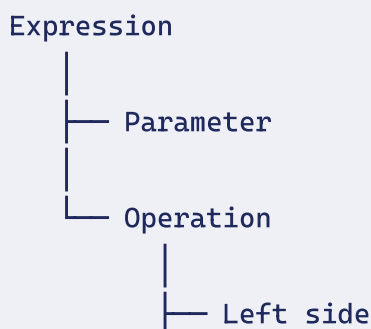
```
"x > 10"
```

لكن الـ Expression Tree مش string.

هي **object graph**.

يعني مجموعة objects مترابطة ببعض.

Conceptually:



فالنظام يقدر يسأل :

- parameter اسميه إيه؟
- نوعه إيه؟
- operation إيه؟
- left side إيه؟
- right side إيه؟
- هل فيه method call؟
- هل فيه property access؟
- هل فيه constant؟

وده اللي يخلي ال-expression قابلة للفحص.

4. أول Expression Tree

نكتب :

```
using System.Linq.Expressions;

Expression<Func<int, bool>> expression =
    x => x > 10;
```

النوع :

```
Expression<Func<int, bool>>
```

معناه بشكل مبسط :

```
Expression
  ↓
  دي delegate shape
  ↓
  Func<int, bool>
```

يعني ال-generic argument بيحدد شكل ال-lambda :

```
input  = int
output = bool
```

لكن بدل ما نكتفي بـ `delegate executable`، إحنا عندنا `expression representation`.

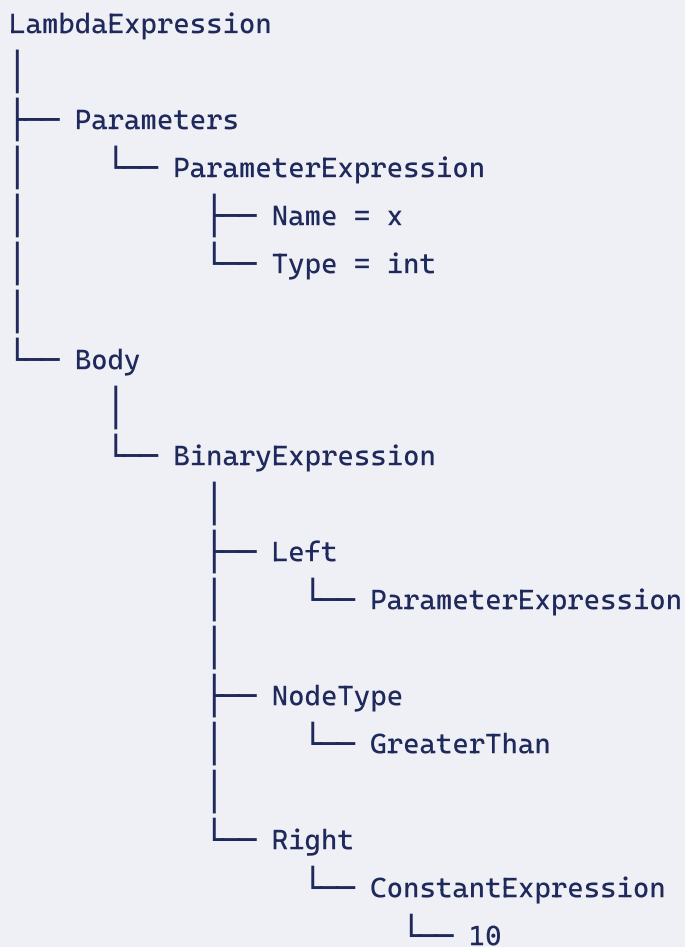
5. الـ Tree فعلاً شكلها عامل إزاي؟

خلينا نمسك:

```
x => x > 10
```

ونفكها.

Conceptually:



خد بالك من حاجة مهمة:

إحنا مش بنمثل:

```
"x > 10"
```

كـstring.

إحنا بنمثّل:

```
x  
>  
10
```

كـnodes مترابطة.

6. الـLambda نفسها Node

ناس كتير بتسمع "Expression Tree" وتتخيل إن الـtree عبارة عن **Body** بس.

لأ.

الـlambda نفسها لها representation.

في:

```
x => x > 10
```

عندنا:

```
LambdaExpression
```

جواها:

```
Parameters
```

```
Body
```

يعني:

```
Lambda  
├── Parameters  
│   └── x  
└──  
    |  
    |  
    |
```

```
└─ Body
  └─ x > 10
```

فـ `LambdaExpression` هي الـ root في الصورة اللي بنبنيتها ذهنياً هنا.

7. ParameterExpression

الـ `x` في:

```
x => x > 10
```

مش مجرد حرف.

هو object بيمثل parameter.

```
var parameter = expression.Parameters[0];

Console.WriteLine(parameter.Name);
Console.WriteLine(parameter.Type);
```

Conceptually:

```
ParameterExpression
├─ Name = "x"
└─ Type = int
```

وده مهم لأن أي system بيفحص الـ tree يقدر يعرف:

"عندي parameter اسمه `x` ونوعه `int`."

8. ConstantExpression

طيب والـ 10 ؟

برضه مش مجرد `text`.

هي `node`:

```
ConstantExpression
```

ولو وصلنا للـ body:

```
var binary =  
    (BinaryExpression)expression.Body;
```

فالـ right side هي الـ constant:

```
var constant = binary.Right;
```

Conceptually:

```
ConstantExpression  
├─ Value = 10  
└─ Type = int
```

فالـ tree شايلة القيمة نفسها.

9. BinaryExpression

دلوقتي أهم جزء:

```
x > 10
```

دي عملية binary operation.

عندنا operand شمال:

```
x
```

وoperand يمين:

```
10
```

وعندنا operator:

```
>
```

والـ Expression API يتمثل ده كـ:

BinaryExpression

وفيها conceptually :

BinaryExpression

```
├── Left
├── NodeType
└── Right
```

والـ **NodeType** هنا :

GreaterThan

يعني :

BinaryExpression

```
├── Left → x
├── Operator → GreaterThan
└── Right → 10
```

10. لما الـ expression تكبر

دلوقتي :

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18;
```

إيه المختلف؟

الـ left side بقى :

c.Age

مش :

c

فدخل عندنا نوع node جديد :

MemberExpression

Conceptually:

```
MemberExpression
├── Member = Age
└── Expression
    └── ParameterExpression(c)
```

يعني :

```
c.Age
├── Member = Age
└── Object = c
```

وده مهم جدًا لأن provider يقدر يفهم إنك بتقرأ property معينة.

11. MethodCallExpression

شوف :

```
Expression<Func<Customer, bool>> expression =
    c => c.Name.Contains("a");
```

هنا عندنا method call .

tree conceptually هتبقى :

```
MethodCallExpression
├── Object
│   ├── MemberExpression
│   │   └── c.Name
└── Arguments
```

```
└─ ConstantExpression
   └─ "a"
```

يعني tree تقدر تقول:

فيه method اسمها `Contains`.

اتنادت على `c.Name`.

وخذت argument قيمته `"a"`.

وده بالضبط النوع من المعلومات اللي provider ممكن يحاول يترجمه.

12. أهم Node Types اللي محتاج تعرفها

مش المطلوب تحفظهم كـ dictionary.

المطلوب تفهم إن أجزاء مختلفة من الـ C# expression ليها representations مختلفة.

أهمهم:

```
LambdaExpression
ParameterExpression
ConstantExpression
BinaryExpression
MemberExpression
MethodCallExpression
UnaryExpression
NewExpression
ConditionalExpression
```

مثلاً:

x

→ `ParameterExpression`

10

→ `ConstantExpression`

```
x > 10
```

→ BinaryExpression

```
c.Age
```

→ MemberExpression

```
c.Name.Contains("a")
```

→ MethodCallExpression

الفكرة مش الحفظ.

الفكرة إنك لما تشوف expression تعرف تسأل:

"إيه الـ nodes اللي ممكن تكون مستخدمة عشان تمثلها؟"

13. Expression Tree مش SQL

دي نقطة مهمة جدًا.

لو عندك:

```
c => c.Age > 18
```

الـ Expression Tree مش معناها:

```
WHERE Age > 18
```

لأ.

هي تمثل:

```
C# expression structure
```

وبعدين provider ممكن يحول الـ structure دي إلى لغة أو representation ثانية.

يعني:

Expression Tree



Provider



Translation

في EF Core غالبًا النتيجة SQL.

لكن tree نفسها مش SQL.

14. طب EF Core بيستفيد منها إزاي؟

هنا ندخل في الـ Backend context.

لما تعمل query على مصدر interpreted زي **IQueryable**، الـ operators الخاصة بالـ queryable بتتعامل مع expressions.

بشكل مبسط:

IQueryable<T>



Queryable.Where(...)



Expression<Func<T, bool>>



Expression Tree



Query Provider



Translation

فبدل ما provider يستلم:

"ده delegate نفدلي الـ"

يقدر يستلم representation للـ query ويبدأ يفحصها.

وده هو السبب الأساسي إن provider يقدر يحاول يفهم:

c => c.Age > 18

15. Func vs Expression<Func<...>>

خلينا نخطهم جنب بعض.

الحالة الأولى

```
Func<Customer, bool> predicate =  
    c => c.Age > 18;
```

mental model:

```
Lambda  
  ↓  
Delegate  
  ↓  
Executable behavior  
  ↓  
CLR executes it
```

الحالة الثانية

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18;
```

mental model:

```
Lambda  
  ↓  
Expression Tree  
  ↓  
Structured representation  
  ↓  
Another system can inspect it
```

ودي واحدة من أهم distinctions في LINQ.

16. فيه IEnumerable عادةً بتعامل مع Func ؟

في LINQ local، المصدر عبارة عن objects موجودة عندك في memory.

زي:

```
List<Customer> customers;
```

لما تعمل:

```
customers.Where(  
    c => c.Age > 18  
);
```

أنت في النهاية محتاج تنفذ condition على كل object.

يعني:

```
Customer  
  ↓  
predicate(customer)  
  ↓  
true / false
```

فالـ delegate مناسب جدًا.

17. ولية IQueryable محتاجة Expression؟

في interpreted query، إحنا مش بالضرورة عايزين نشغل الـ condition داخل CLR على كل object.

إحنا عايزين provider يفهم الـ condition.

مثلاً:

```
query.Where(  
    c => c.Age > 18  
);
```

الـ provider ممكن يشوف:

```
Parameter: c  
Property: Age
```

Operator: GreaterThan
Constant: 18

وبناءً على ده يترجمها.

يعني:

```
C# structure
  ↓
Expression Tree
  ↓
Provider understands structure
  ↓
Translation
```

18. طب لو استخدمت Method عادية؟

مثلاً:

```
bool IsAdult(Customer customer)
{
    return customer.Age > 18;
}
```

ومع local data:

```
customers.Where(IsAdult);
```

مفيش مشكلة.

لأن LINQ local محتاجة executable delegate.

لكن في provider، interpreted query محتاج يعرف structure اللي جوه الـ method.

لو استلم مجرد method call:

```
IsAdult(customer)
```

فهو مش بالضرورة عنده implementation الداخلية:


```
customer.Age > 18
```

expression trees قابلة للتحليل.

وده سبب مهم يخليك تفهم الفرق بين :

```
"function" نفذلي
```

و:

```
"function logic" اديني structure
```

تقدر تفحصها Expression Tree 19.

وده الجزء اللي يخلي الموضوع ممتع.

عندك :

```
Expression<Func<int, bool>> expression =  
    x => x > 10;
```

ممکن تعمل :

```
Console.WriteLine(expression.NodeType);  
Console.WriteLine(expression.Parameters.Count);  
Console.WriteLine(expression.Body.NodeType);
```

وتبدأ تستكشف.

مثلاً :

```
var parameter = expression.Parameters[0];  
  
Console.WriteLine(parameter.Name);  
Console.WriteLine(parameter.Type);
```

وبعدين :

```
var binary =  
    (BinaryExpression)expression.Body;
```

```
Console.WriteLine(binary.NodeType);
Console.WriteLine(binary.Left.NodeType);
Console.WriteLine(binary.Right.NodeType);
```

إنت هنا حرفيًا بتقرأ الـ object graph.

20. ExpressionVisitor

لو عندك tree كبيرة، مش منطقي تكتب:

```
body
  ↓
left
  ↓
left
  ↓
...
```

بنفسك كل مرة.

هنا بيظهر:

ExpressionVisitor

وده abstraction للـ traversal.

مثال:

```
class MyVisitor : ExpressionVisitor
{
    protected override Expression VisitBinary(
        BinaryExpression node)
    {
        Console.WriteLine(
            $"Binary: {node.NodeType}");

        return base.VisitBinary(node);
    }
}
```

وبعدين :

```
var visitor = new MyVisitor();  
  
visitor.Visit(expression);
```

الفكرة :

```
Root  
↓  
Visit  
↓  
Child  
↓  
Child  
↓  
Child
```

21. **ليه ExpressionVisitor** مهم هندسيًا؟

لأننا مش بنستخدم الـ tree بس عشان نبص عليها.

ممکن نعمل :

```
Inspect  
Validate  
Transform  
Rewrite  
Analyze
```

مثلاً ممكن visitor يكتشف :

هل الـ query فيها method معينة؟

أو :

هل فيه comparison معين؟

أو :

هل فيه property معينة؟

أو حتى يعمل transformation.

وده بيدخلنا في عالم أكبر:

Expression Processing

22. ممكن نبني Expression Tree بإيدنا

ودي واحدة من أحسن الحاجات اللي تثبتك إن الموضوع مش magic.

عايزين نمثل:

```
x > 10
```

نبدأ بال-parameter:

```
var parameter =  
    Expression.Parameter(  
        typeof(int),  
        "x");
```

ثم constant:

```
var constant =  
    Expression.Constant(10);
```

ثم binary operation:

```
var body =  
    Expression.GreaterThan(  
        parameter,  
        constant);
```

ثم lambda:

```
var lambda =  
    Expression.Lambda<Func<int, bool>>(  
        body,  
        parameter);
```

إحنا حرفيًا بنبني:

```
Parameter
  +
Constant
  ↓
GreaterThan
  ↓
Lambda
```

وده يثبت إن Expression Tree عبارة عن structure من nodes.

23. طب نقدر نشغلها؟

آه.

ودي نقطة جميلة جدًا.

عندك:

```
Expression<Func<int, bool>> expression =
    x => x > 10;
```

ممکن تعمل:

```
Func<int, bool> compiled =
    expression.Compile();
```

وبعدين:

```
var result = compiled(15);
```

يعني:

```
Expression Tree
  ↓
Compile()
  ↓
Delegate
```

↓
Execution

لاحظ الفرق:

الـ Expression Tree كانت representation.

executable delegate حولتها إلى `Compile()`.

24. يبقى Expression Tree مش Execution

دي misconception لازم تموت.

لما تعمل:

```
Expression<Func<int, bool>> expression =  
    x => x > 10;
```

أنت مش بتنفذ:

```
x > 10
```

أنت بتخزن representation.

لو عايز execution محلي:

```
var func = expression.Compile();  
  
func(15);
```

لكن لو provider هو اللي هيستخدمها:

```
Expression Tree  
    ↓  
Provider  
    ↓  
Translation / Processing
```

tree نفسها مش execution engine.

25. Expression Tree وDynamic Queries

هنا بقي نقدر نستفيد من فكرة إن الـ expression عبارة عن objects.

افتراض عندك search screen :

```
Age > 18
Name contains "Moh"
Salary > 10000
```

بدل ما تكتب كل query combination يدويًا، ممكن application تبني expressions dynamically.

Conceptually:

```
User Filters
  ↓
Build Expression Tree
  ↓
Combine Expressions
  ↓
Query Provider
  ↓
Translation
```

وده مفيد في:

- dynamic filtering
- search
- reporting
- specification-style queries
- reusable filtering infrastructure

لكن لازم تستخدمه بحكمة.

26. أهم نقطة: Provider هو اللي بفهم

ما نقولش:

"طالما عندي Expression Tree يبقى أي حاجة أكتبها هتتحول SQL."

لا.

Expression Tree بتدي provider **structure**.

لكن provider هو اللي بيقرر:

إيه اللي أقدر أترجمه؟

مثال:

```
c => c.Age > 18
```

واضحة جدًا.

لكن:

```
c => MyCrazyBusinessRule(c)
```

مش معناها إن EF Core يقدر تلقائيًا يحول implementation الداخلية للـ method إلى SQL.

فـ mental model الصحيح:

Expression Tree

↓

"Here is the structure."

↓

Provider

↓

"Here is what I know how to translate."

27. Expression Tree Parser للـ C# Source Code

برضه misconception.

إحنا مش بندي provider:

```
"x => x.Age > 18"
```

كـ string.

الـ compiler هو اللي بيكوّن representation.

وبعدين الـ runtime object model بتاعت Expression API بتخلينا نتعامل مع structure دي.

فبديل:

Raw text

↓

Parse text

عندنا:

C# code

↓

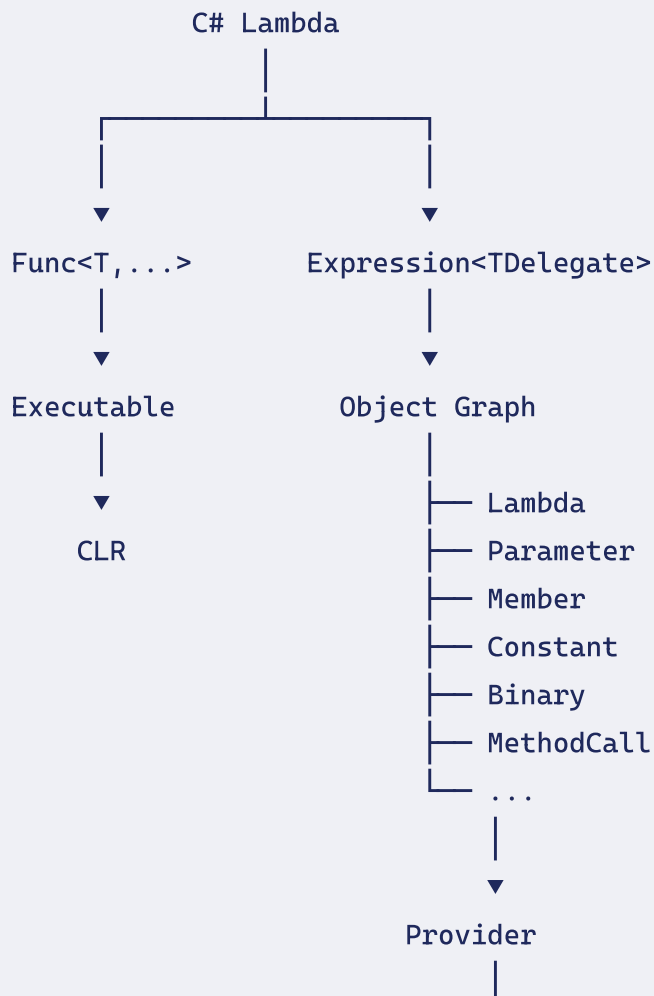
Compiler

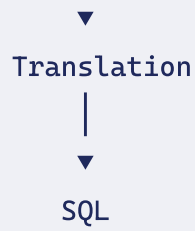
↓

Expression representation

28. الصورة الكاملة

دلوقتي اجمع كل حاجة:





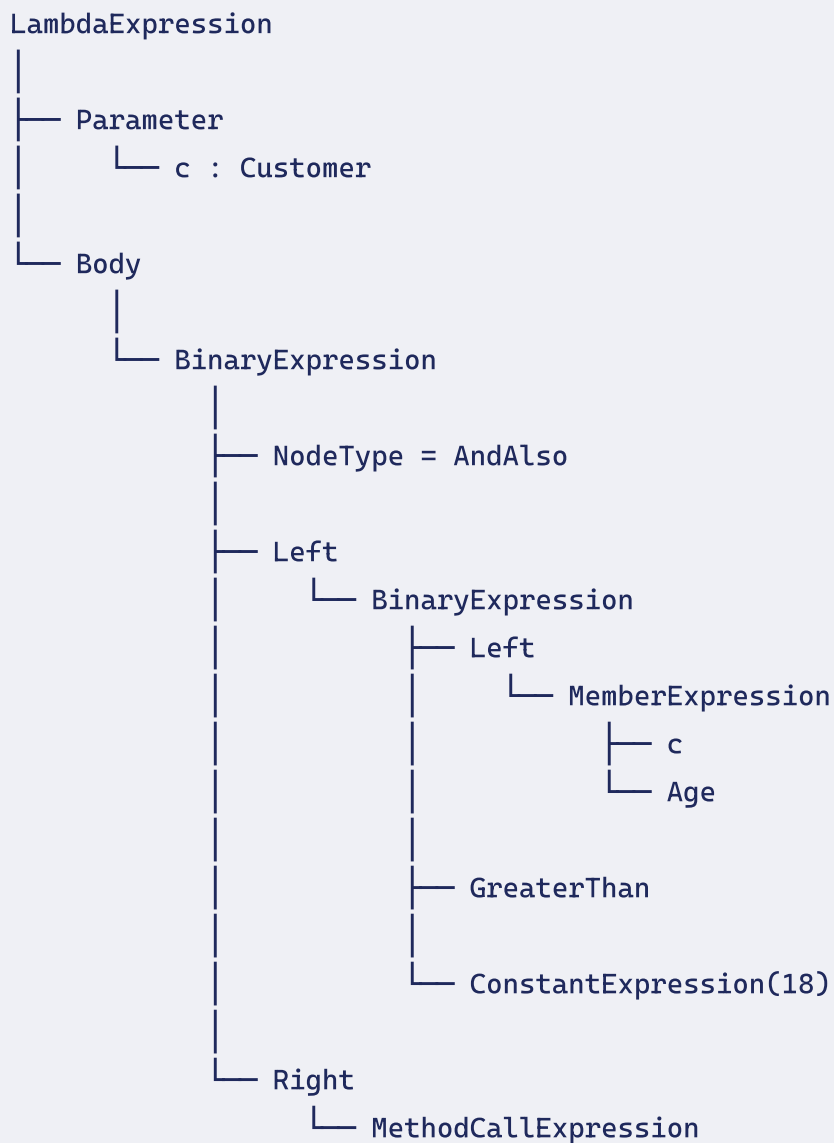
لو فهمت الرسم ده، إنت فهمت قلب الموضوع.

29. مثال كامل من البداية للنهاية

خلينا نعمل:

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18 && c.Name.Contains("Moh");
```

يمكن تتخيل الـ tree:



```
c.Name  
"Moh"
```

وده مثال ممتاز يوضح ليه اسمها **Tree**.

عندك root، وبعدها branches، وبعدها leaves.

30. ليه الموضوع ده مهم ليك كـ Backend Engineer؟

لأنك لو بتستخدم:

EF Core
LINQ
Specification Pattern
Dynamic Filtering
Query Providers

فأنت بتتعامل مع concept مهم جداً حتى لو مش بتكتبه بإيدك كل يوم.

وأهم من كده:

بتعلمك طريقة تفكير مهمة Expression Trees:

الكود مش لازم يكون حاجة تنفذ فقط؛ ممكن يكون حاجة يتم تمثيلها وتحليلها وتحويلها.

وده concept موجود في حاجات كتير في عالم software engineering:

Compiler
Parser
AST
Query Translator
Rule Engine
Expression Evaluator
Code Generation

31. أخطاء شائعة

الخطأ 1

Expression Tree = SQL

غلط.

هي representation لل-expression.

الخطأ 2

بتتنفذ تلقائيًا Expression Tree

غلط.

يمكن Compile() أو provider يعالجها.

الخطأ 3

أي C# method تنفع داخل EF Core query

مش بالضرورة.

الـ provider له translation capabilities.

الخطأ 4

string مجرد Expression Tree

غلط.

هي typed object graph.

الخطأ 5

نفس الحاجة <>Expression<Func> و Func

غلط.

الـ semantic model مختلف: syntax، لكن

Func

→ executable behavior

Expression<Func<>>

→ representation of behavior

32. Final Mental Model EG

لو عايز تحفظ جملة واحدة:

قابل **behavior** تتحول من مجرد **C# expression** هي طريقة نخلي بيها الـ **Expression Tree** قابلة للفحص والتحليل والتحويل **structure** للتنفيذ إلى.

وبالتالي:

```
c => c.Age > 18
```

مش لازم معناها فقط:

"خذ c وشوف هل عمره أكبر من 18."

ممکن معناها:

"دي structure فيها parameter اسمه c، و property اسمها Age، و operator اسمه GreaterThan، و constant قيمته 18."

والفرق ده هو اللي بيفتح الباب للـ query providers.

Complete English Explanation

1. What Are We Actually Trying to Understand?

Consider:

```
c => c.Age > 18
```

At first glance, it is simply a lambda expression.

You might say:

```
"It takes a customer and returns whether the customer is older than 18."
```

That is correct, but it is not the whole story.

The same lambda syntax can participate in two fundamentally different models:

```
Func<Customer, bool> predicate =  
    c => c.Age > 18;
```

or:

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18;
```

The first represents executable behavior.

The second represents the structure of that behavior.

That distinction is the foundation of Expression Trees.

2. **Func** Means "Execute"; **Expression** Means "Represent"

With:

```
Func<int, bool> predicate =  
    x => x > 10;
```

you can execute it:

```
var result = predicate(15);
```

The runtime executes the represented behavior.

With:

```
Expression<Func<int, bool>> expression =  
    x => x > 10;
```

the lambda is represented as a structured object model.

The conceptual distinction is:

```
Func  
↓  
Executable behavior  
  
Expression<Func<...>>  
↓  
Structured representation
```

This difference enables inspection and translation.

3. What Does "Representation" Mean?

An Expression Tree is not simply:

```
"x > 10"
```

stored as a string.

It is a typed object graph.

Conceptually:

```
Expression  
├── Parameter  
└── Operation  
    ├── Left  
    └── Right
```

This means another component can inspect the expression and ask:

- What parameters exist?

- What are their types?
 - What operation is being performed?
 - What is the left operand?
 - What is the right operand?
 - Is a method being called?
 - Is a property being accessed?
 - What constant values exist?
-

4. Creating an Expression Tree

```
using System.Linq.Expressions;

Expression<Func<int, bool>> expression =
    x => x > 10;
```

The generic argument describes the delegate shape:

```
Input  → int
Output → bool
```

But `Expression<TDelegate>` represents the expression rather than merely providing an executable delegate.

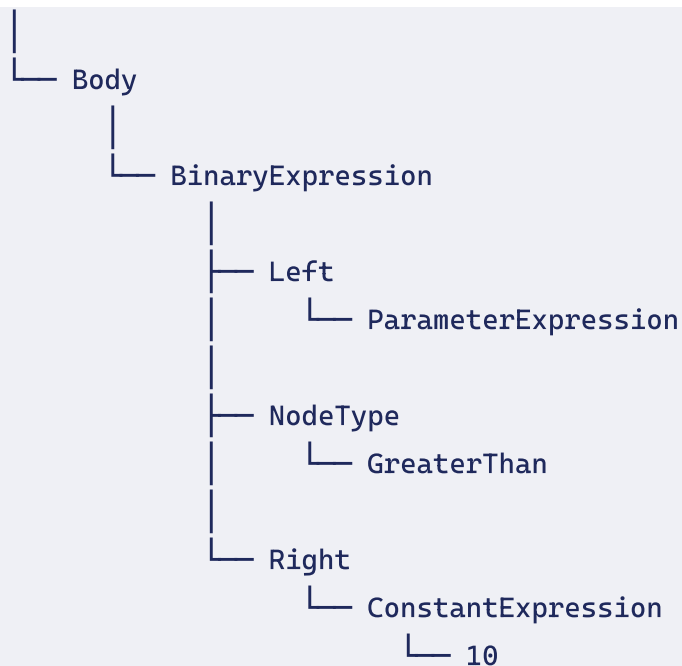
5. What Does the Tree Look Like?

For:

```
x => x > 10
```

a useful conceptual representation is:

```
LambdaExpression
├── Parameters
│   └── ParameterExpression
│       ├── Name = x
│       └── Type = int
```

The expression is therefore a hierarchy of typed nodes.

6. LambdaExpression

The lambda itself has a representation.

For:

```
x => x > 10
```

we can conceptually see:

```
LambdaExpression
├── Parameters
│   └── x
└── Body
    └── x > 10
```

The lambda contains the parameters and its body.

7. ParameterExpression

The parameter:

```
x
```

is represented by a `ParameterExpression` .

You can inspect it:

```
var parameter = expression.Parameters[0];  
  
Console.WriteLine(parameter.Name);  
Console.WriteLine(parameter.Type);
```

Conceptually:

```
ParameterExpression  
├─ Name = "x"  
└─ Type = int
```

8. `ConstantExpression`

The literal:

```
10
```

is represented by a `ConstantExpression` .

If the body is binary:

```
var binary =  
    (BinaryExpression)expression.Body;
```

then the right operand represents the constant.

Conceptually:

```
ConstantExpression  
├─ Value = 10  
└─ Type = int
```

9. BinaryExpression

The operation:

```
x > 10
```

is represented by a `BinaryExpression`.

It has:

```
Left  
NodeType  
Right
```

Conceptually:

```
BinaryExpression  
├─ Left → x  
├─ NodeType → GreaterThan  
└─ Right → 10
```

The operator is represented structurally rather than stored as raw source text.

10. MemberExpression

Consider:

```
c => c.Age > 18
```

The expression:

```
c.Age
```

is represented as member access.

Conceptually:

```
MemberExpression  
├─ Member = Age
```

```
└─ Expression
    └─ ParameterExpression(c)
```

This allows a provider to understand that a specific property is being accessed.

11. MethodCallExpression

Consider:

```
c => c.Name.Contains("a")
```

Conceptually:

```
MethodCallExpression
├── Object
│   └── MemberExpression
│       └── c.Name
└── Arguments
    └── ConstantExpression
        └── "a"
```

The tree can therefore represent:

- the method
- the target object
- the arguments

That structure can then be inspected by another component.

12. Important Expression Node Types

Some important node types include:

```
LambdaExpression
ParameterExpression
ConstantExpression
```

```
BinaryExpression
MemberExpression
MethodCallExpression
UnaryExpression
NewExpression
ConditionalExpression
```

The goal is not to memorize a list.

The goal is to recognize that different parts of a C# expression are represented by different node types.

13. Expression Trees Are Not SQL

An Expression Tree such as:

```
c => c.Age > 18
```

is not:

```
WHERE Age > 18
```

It represents the structure of the .NET expression.

A provider may translate that structure into SQL:

```
Expression Tree
  ↓
Provider
  ↓
Translation
  ↓
SQL
```

The Expression Tree itself is not SQL.

14. How Does EF Core Benefit?

For interpreted LINQ queries, queryable operators work with expression trees.

A simplified model is:

```
IQueryable<T>
  ↓
Queryable.Where(...)
  ↓
Expression<Func<T, bool>>
  ↓
Expression Tree
  ↓
Query Provider
  ↓
Translation
```

Instead of receiving only executable behavior, the provider can inspect a structured representation of the query.

That is what makes provider-side query translation possible.

15. **Func** vs **Expression<Func<...>>**

Executable model

```
Func<Customer, bool> predicate =
    c => c.Age > 18;
```

Mental model:

```
Lambda
  ↓
Delegate
  ↓
Executable behavior
  ↓
CLR
```

Representation model

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18;
```

Mental model:

```
Lambda  
  ↓  
Expression Tree  
  ↓  
Structured representation  
  ↓  
Inspection / transformation / translation
```

The syntax looks similar, but the underlying model is different.

16. Why Local LINQ Uses Delegates

For local data:

```
List<Customer> customers;
```

a query such as:

```
customers.Where(  
    c => c.Age > 18  
);
```

needs to evaluate the predicate against objects already available in memory.

Conceptually:

```
Customer  
  ↓  
predicate(customer)  
  ↓  
true / false
```

An executable delegate is exactly what is needed.

17. Why Provider-Based LINQ Uses Expressions

With an interpreted query, the provider may want to understand the condition rather than execute it against each object in the current process.

For:

```
query.Where(  
    c => c.Age > 18  
);
```

the provider can conceptually see:

```
Parameter: c  
Property: Age  
Operator: GreaterThan  
Constant: 18
```

and attempt to translate that structure.

18. Why Ordinary Methods Can Be a Problem

Suppose you have:

```
bool IsAdult(Customer customer)  
{  
    return customer.Age > 18;  
}
```

For local LINQ:

```
customers.Where(IsAdult);
```

this is fine because the delegate can simply execute the method.

For a query provider, however, the provider needs to understand the logic.

If all it sees is:


```
IsAdult(customer)
```

it does not automatically receive the method implementation as a traversable expression tree.

That is a key difference between:

```
Execute this behavior
```

and:

```
Give me the structure of this behavior
```

19. Inspecting an Expression Tree

Given:

```
Expression<Func<int, bool>> expression =  
    x => x > 10;
```

you can inspect:

```
Console.WriteLine(expression.NodeType);  
Console.WriteLine(expression.Parameters.Count);  
Console.WriteLine(expression.Body.NodeType);
```

Then:

```
var parameter = expression.Parameters[0];  
  
Console.WriteLine(parameter.Name);  
Console.WriteLine(parameter.Type);
```

and:

```
var binary =  
    (BinaryExpression)expression.Body;  
  
Console.WriteLine(binary.NodeType);
```

```
Console.WriteLine(binary.Left.NodeType);
Console.WriteLine(binary.Right.NodeType);
```

You are effectively navigating the object graph.

20. ExpressionVisitor

For larger trees, manually navigating every node becomes impractical.

.NET provides:

```
ExpressionVisitor
```

Example:

```
class MyVisitor : ExpressionVisitor
{
    protected override Expression VisitBinary(
        BinaryExpression node)
    {
        Console.WriteLine(
            $"Binary: {node.NodeType}");

        return base.VisitBinary(node);
    }
}
```

Then:

```
var visitor = new MyVisitor();

visitor.Visit(expression);
```

The visitor recursively traverses the tree.

21. Why ExpressionVisitor Matters

Expression Trees are not useful only because we can inspect them.

We can also:

```
Inspect
Validate
Analyze
Transform
Rewrite
```

This makes expression processing useful in systems involving:

- query translation
- dynamic filtering
- rule engines
- specification-style abstractions
- expression rewriting

The tree becomes a manipulatable representation of logic.

22. Building an Expression Tree Manually

We can represent:

```
x > 10
```

by constructing its nodes.

Parameter:

```
var parameter =
    Expression.Parameter(
        typeof(int),
        "x");
```

Constant:

```
var constant =
    Expression.Constant(10);
```

Binary operation:

```
var body =  
    Expression.GreaterThan(  
        parameter,  
        constant);
```

Lambda:

```
var lambda =  
    Expression.Lambda<Func<int, bool>>(  
        body,  
        parameter);
```

Conceptually:

```
Parameter  
  +  
Constant  
  ↓  
GreaterThan  
  ↓  
Lambda
```

This demonstrates that an Expression Tree is an object structure rather than magic.

23. Compiling an Expression Tree

An Expression Tree can be converted back into executable behavior:

```
Expression<Func<int, bool>> expression =  
    x => x > 10;  
  
Func<int, bool> compiled =  
    expression.Compile();  
  
var result = compiled(15);
```

The conceptual flow is:

Expression Tree



Compile()



Delegate



Execution

So an Expression Tree can serve as an intermediate representation that can later be compiled.

24. An Expression Tree Does Not Execute Automatically

Creating:

```
Expression<Func<int, bool>> expression =  
    x => x > 10;
```

does not mean:

```
x > 10
```

has already executed.

The expression is a representation.

For local execution:

```
var func = expression.Compile();  
  
func(15);
```

For provider-based processing:

Expression Tree



Provider



Translation / Processing

The tree itself is not an execution engine.

25. Dynamic Queries

Because Expression Trees are structured objects, applications can build expressions dynamically.

For example:

```
Age > 18
Name contains "Moh"
Salary > 10000
```

can conceptually become:

```
User Filters
  ↓
Expression Tree Construction
  ↓
Expression Composition
  ↓
Query Provider
  ↓
Translation
```

This is useful for:

- dynamic filtering
- search
- reporting
- reusable specifications
- generic query infrastructure

But excessive expression abstraction can make code difficult to understand, so it should be used deliberately.

26. The Provider Is the Translator

A critical distinction:

An Expression Tree does not guarantee SQL translation.

The tree provides structure.

The provider decides what it understands and how it can translate it.

For example:

```
c => c.Age > 18
```

may be straightforward.

But:

```
c => MyBusinessRule(c)
```

does not automatically mean the provider can translate the internal implementation of `MyBusinessRule` into SQL.

The correct mental model is:

```
Expression Tree
  ↓
"Here is the structure."
  ↓
Provider
  ↓
"Here is what I know how to translate."
```

27. Expression Trees Are Not Raw C# Source

An Expression Tree is not simply source code stored as:

```
"x => x.Age > 18"
```

The compiler creates the appropriate expression representation.

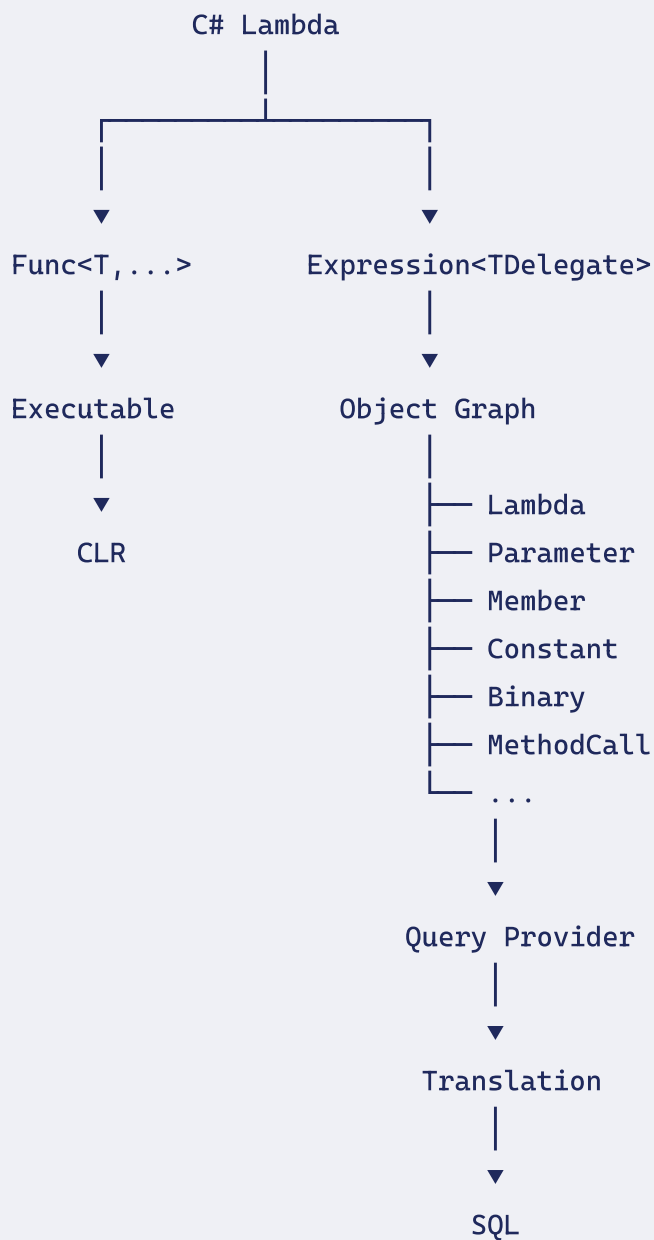
The runtime API then exposes that structure through typed objects.

Conceptually:

```
C# Source
  ↓
Compiler
  ↓
Expression Representation
```

This is fundamentally different from passing raw source text around.

28. Complete Mental Model



This diagram captures the central idea:

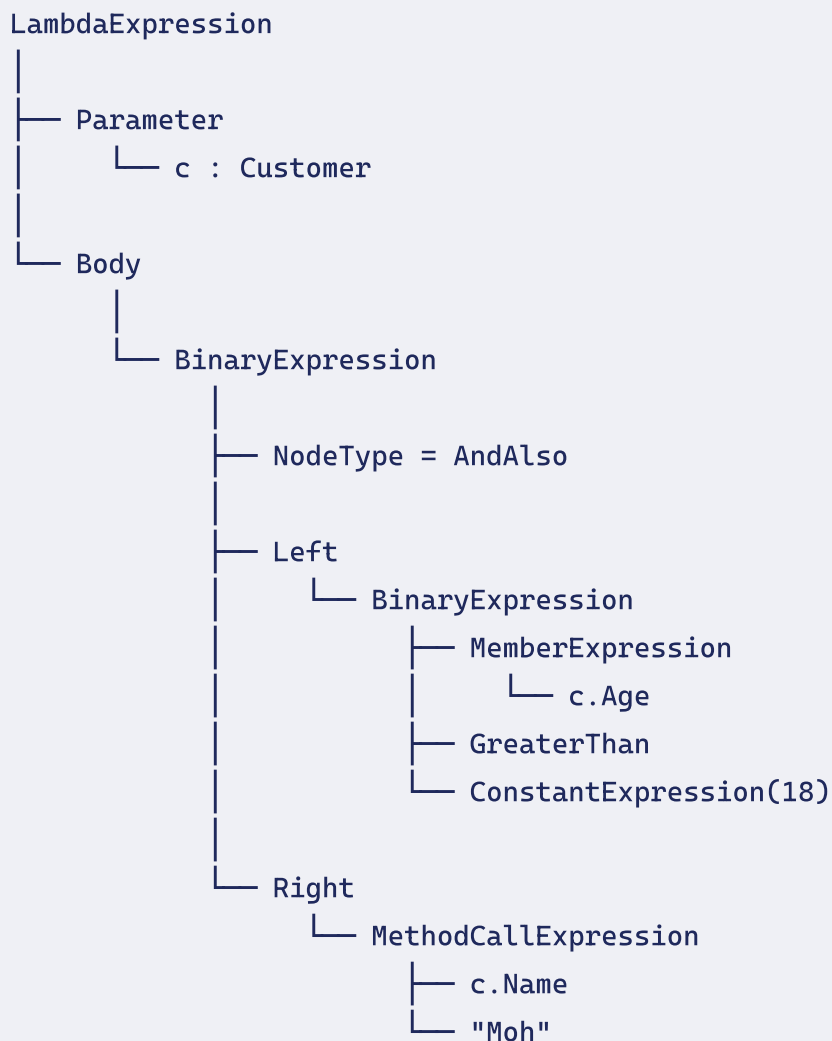
A lambda can represent executable behavior, or it can become a structured representation that another system can inspect and process.

29. Full Example

Consider:

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18 && c.Name.Contains("Moh");
```

A useful conceptual representation is:



This is why the term **tree** is appropriate.

There is a root, branches, and child nodes.

30. Why This Matters to a Backend Engineer

Expression Trees matter whenever you work with concepts such as:

- EF Core
- LINQ
- Query Providers
- Dynamic Filtering
- Specification Pattern
- Expression Composition

More importantly, they teach a broader engineering concept:

Code does not always have to be executed immediately. It can be represented as data, inspected, transformed, and interpreted by another system.

That idea appears in many areas of software engineering:

- Compiler
- Parser
- AST
- Query Translator
- Rule Engine
- Expression Evaluator
- Code Generation

Expression Trees are one concrete .NET implementation of that broader idea.

31. Common Misconceptions

Misconception 1

Expression Tree = SQL

No.

It represents the .NET expression structure.

Misconception 2

Creating an Expression Tree executes it.

No.

It represents the expression.

It can later be compiled or processed by a provider.

Misconception 3

Every C# method can be translated by EF Core.

No.

The provider supports a particular translation vocabulary.

Misconception 4

An Expression Tree is a string.

No.

It is a typed object graph.

Misconception 5

`Func<T,...>` and `Expression<Func<T,...>>` are the same.

No.

Their syntax may look similar, but their semantic roles are different:

`Func<T,...>`

→ executable behavior

`Expression<Func<T,...>>`

→ structured representation

32. Final Mental Model

The one sentence to remember is:

An Expression Tree turns a .NET expression from merely executable behavior into a structured representation that can be inspected, analyzed, transformed, or translated.

So:

```
c => c.Age > 18
```

does not have to mean only:

"Take `c` and check whether its age is greater than 18."

In an Expression Tree, it can represent:

```
Parameter: c  
Property: Age  
Operator: GreaterThan  
Constant: 18
```

That distinction is the foundation that allows query providers to reason about LINQ expressions.

Final Challenge

Before considering the topic mastered, explain this without looking at the notes:

```
Expression<Func<Customer, bool>> expression =  
    c => c.Age > 18;
```

1. What is the type of `expression`?
2. What node represents `c`?
3. What node represents `18`?
4. What node represents `c.Age > 18`?
5. Why can a provider inspect this expression?

6. What is the fundamental difference between `Func<Customer, bool>` and `Expression<Func<Customer, bool>>`?
7. What does `ExpressionVisitor` do?
8. Why does an Expression Tree not automatically mean SQL?

If you can answer those questions clearly, you understand the core of Expression Trees rather than simply memorizing the API.